Rails Simply Feels Right

A large number of developers were frustrated with the technologies they were using to create web applications. It didn't seem to matter whether they used Java, PHP, or .NET—there was a growing sense that their jobs were just too damn hard. And then, suddenly, along came Rails, and Rails was easier.

But easy on its own doesn't cut it. We're talking about professional developers writing real-world websites. They wanted to feel that the applications they were developing would stand the test of time—that they were designed and implemented using modern, professional techniques. So, these developers dug into Rails and discovered it wasn't just a tool for hacking out sites.

For example, *all* Rails applications are implemented using the model-view-controller (MVC) architecture. MVC isn't a new concept for web development—the earliest Java-based web frameworks (like Struts) base their design on it. But Rails takes MVC further: when you develop in Rails, you start with a working application, each piece of code has its place, and all the pieces of your application interact in a standard way.

Professional programmers write tests. And again, Rails delivers. All Rails applications have testing support baked right in. As you add functionality to the code, Rails automatically creates test stubs for that functionality. The framework makes it easy to test applications, and, as a result, Rails applications tend to get tested.

Rails applications are written in Ruby, a modern, object-oriented language. Ruby is concise without being unintelligibly terse. You can express ideas naturally and cleanly in Ruby code. This leads to programs that are easy to write and (just as important) easy to read months later.

Rails takes Ruby to the limit, extending it in novel ways that make our programming lives easier. Using Rails makes our programs shorter and more readable. It also allows us to perform tasks that would normally be done in